

Wrap-Up Report

Wrap-Up Report

수능형 문제 풀이 모델 생성

가을(T8001), 박신지(T8078), 박희권(T8091), 이형석(T8164)

I. Introduction

프로젝트 주제.

본 프로젝트는 '수능형 문제 풀이 모델 생성' 대회로, 한국어 수능(국어·사회)과 유사한 지문 기반 객관식 문제를 풀 수 있는 모델을 개발하는 것을 목표로 진행되었습니다. 대회에서는 지문과 질문, 선택지가 주어졌을 때 정답 선택지를 예측해야 하며, 대형 모델뿐 아니라 상대적으로 작은 규모의 모델로도 이러한 시험형 문제를 얼마나 잘 해결할 수 있는지에 초점을 둡니다.

평가 지표.

본 대회는 5개 클래스(정답 레이블 1~5) 다중 분류 문제이며, 최종 순위 산정은 Macro F1-score를 기준으로 이루어집니다. Macro F1-score는 각 클래스별 F1-score를 개별 계산한 뒤 단순 평균한 값으로, 특정 클래스에서만 성능이 높고 다른 클래스에서 성능이 낮은 경우 불리할 수 있는 지표라고 이해했습니다.

따라서 단순 정확도(Accuracy) 향상만 목표로 하기보다, 한쪽으로 치우친 예측을 줄이고 전반적으로 균형 잡힌 정답 예측을 유도하는 방향이 필요하다고 판단했습니다. 다만 대회 특성상 정답이 공개되지 않아 클래스별 성능을 직접 확인할 수는 없었고, public 리더보드 점수 변화를 기준으로 개선 여부를 간접적으로 확인하며 실험을 진행했습니다.

실험 환경.

실험은 NVIDIA Tesla V100 GPU 서버 3대를 사용하여 진행되었습니다. 모든 서버는 SSH 원격 접속 방식을 통해 개발 환경에 접근하여 사용되었습니다.

II. Team Composition

역할.

Table 1. 수능형 문제 풀이 모델 생성 프로젝트 기간 동안 팀원 별 역할

T8001 가을	wikipedia 문서로 RAG 실험
T8078 박신지	DeepSeek Distill Qwen2.5 32B 실험, 공무원 시험 데이터로 증강 시도, Wikipedia api 이용한 RAG
T8091 박희권	베이스 모델 선정, 모델 학습 방법론 실험, 앙상블 조합 탐색
T8164 이형석	CoT, ORPO, RAG, SC 등 다양한 추론 학습 기법 실험

협업 문화.

본 프로젝트는 효율적인 협업, 코드 품질 유지, 그리고 연구 결과의 체계적인 기록 및 공유를 위해 다음과 같은 도구와 컨벤션을 설정하고 활용했습니다.

1. 버전 관리 및 코드 공유

1.1 [GitHub 및 Git Convention](#)

모든 코드 버전 관리는 GitHub를 중심으로 이루어졌으며, 안정적인 개발을 위해 브랜치명(예: ke-01) 및 커밋 메시지 규칙을 설정하여 준수했습니다. 또한 GitHub Projects를 활용하여 태스크를 관리하고, Issue Template을 설정하여 버그 보고 및 기능 요청을 체계화했습니다.

1.2 [Hugging Face](#)

실험된 모델 가중치 및 아티팩트 공유를 위해 Hugging Face Hub(NLP-07-ODQA)를 활용했습니다.

2. 커뮤니케이션 및 문서화

2.1 Notion

프로젝트의 모든 기록, 실험 계획, 결과 분석 및 회의록을 관리했습니다. 특히, 원격 서버의 실시간 상태(GPU, 메모리)를 Notion에서 투명하게 공유하여 자원 활용 효율을 높였습니다.

2.2 Slack

실시간 질의응답 및 공지 등 신속한 커뮤니케이션을 위한 주요 채널로 활용되었습니다.

III. Process

프로젝트 수행 기간 및 활동내용.

Table 2. 프로젝트 기간 별 수행 내역

	수행내용	날짜
A	프로젝트 준비	
	강의 수강 및 실험 계획 수립	12/15~19
	원격 서버 생성 및 세팅	12/17~19
	협업을 위한 깃허브 세팅 (브랜치명, 이슈 템플릿, 깃허브 프로젝트)	12/18
	모델 공유를 위한 Hugging Face 레포 생성	12/18
B	데이터셋 관련	
	데이터셋 EDA 수행	12/15~16

	정답 클래스 불균형 해소	12/28
	문제 오류, 라벨 이상 데이터 정제	12/29~30
	공무원 시험 데이터 증강	1/3
C	학습 관련 실험	
	Unsloth 베이스라인 구축	12/17~20
	Pre-trained 모델 성능 비교	12/22~25
	CoT 데이터 학습 실험	12/24~29
	파라미터 튜닝	12/26~30
D	추론 관련 실험	
	Self-Consistency 추론	12/22~29
	Wikipedia 데이터 RAG 활용 추론	12/29~1/6
	Llama Cpp 활용 Zero-shot 추론	1/5~6
E	양상블 관련 실험	
	확률 평균 기반 양상블	12/29~1/6

IV. Experiments

Dataset.

학습 데이터는 총 2,031개이며, 'id', 'paragraph', 'problems', 'question_plus' 4개 컬럼으로 구성되어 있습니다. 'id'는 샘플을 구분하는 고유 식별자이고, 'paragraph'는 문제 풀이에 필요한 지문입니다. 'problems'에는 문항 정보가 딕셔너리 형태로 포함되며 'question'(질문), 'choices'(선택지), 'answer'(정답 인덱스)로 구성되어 있습니다.

'question_plus'는 보거나 추가 조건 등 부가 정보를 담고 있으며, 지문 및 문항과 함께 최종 입력을 구성할 때 활용됩니다.

평가 데이터는 총 869개이며, 수능형 문제와 KMMLU / MMMLU(Ko) / KLUE MRC 데이터로 구성되어 있습니다. 평가 데이터의 정답은 빈 문자열로 제공되어 추론 결과 제출 방식으로 평가가 진행됩니다. 또한 전체 test.csv 파일은 public 434개와 private 435개로 랜덤 분할되어 구성되어 있습니다.

EDA.

프로젝트 수행에 앞서 EDA를 진행하여 데이터의 특성, 품질, 분포 등을 파악하였습니다.

1. 결측치 분석

EDA 과정에서 'question_plus' 컬럼이 학습 데이터에서는 대부분 비어 있었고, 테스트 데이터에서도 전체 869개 중 44개 샘플에만 존재하는 등 전반적으로 희소하다는 점을 확인하였습니다. 이러한 특성으로 인해 해당 컬럼만을 대상으로 별도의 데이터 증강이나 추가 전처리 규칙을

설계하기에는 근거가 부족하다고 판단하였습니다. 대신 값이 존재하는 경우에는 이를 질문과 선택지 사이에 '<보기>' 형태로 포함해 입력을 구성하면, 모델이 문맥상 필요한 정보를 자연스럽게 활용할 수 있을 것으로 보았습니다. 따라서 본 프로젝트에서는 추가적인 보정/증강 없이, 입력 포맷에 포함하는 방식으로만 처리하였습니다.

2. 문제 길이 분포

EDA 과정에서 문제 길이 분포를 확인한 결과, 학습 데이터는 대체로 중간 길이 구간에 집중되어 있었고 극단적으로 긴 문제는 상대적으로 드물었습니다. 반면 테스트 데이터에서는 대부분이 짧은 구간에 분포하였으나 일부 샘플에서 매우 긴 문제가 존재하는 롱테일 형태를 보였습니다. 이에 따라 추론 단계에서 입력 길이로 인한 정보 손실 가능성을 줄이기 위해 max_tokens를 4096으로 여유 있게 설정하여 실험을 진행하였습니다.

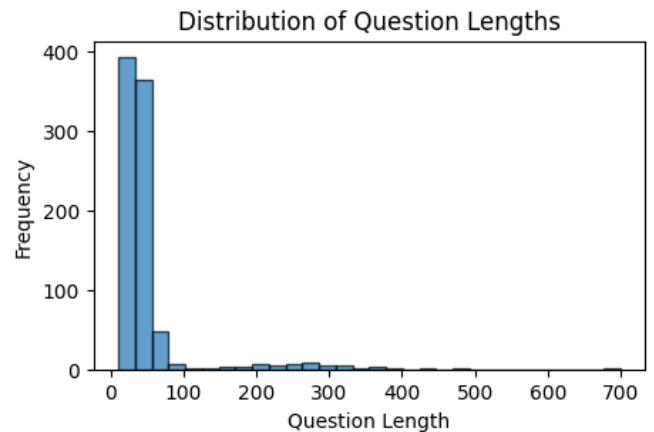


Figure 1. 테스트 데이터 문제 길이 분포

3. 정답 클래스 불균형

EDA 과정에서 학습 데이터의 정답 레이블 분포를 확인한 결과, 1번 정답이 948개로 가장 많고 5번 정답이 31개로 가장 적어 클래스 간 데이터 수가 크게 차이 나는 불균형이 존재하였습니다. 이러한 분포는 모델이 다수 클래스에 편향된 예측을 수행할 가능성이 있어, 불균형이 실제 성능에 영향을 미치는지 확인할 필요가 있다고 판단하였습니다. 이에 따라 클래스 불균형의 영향 여부를 점검하기 위한 후속 실험을 진행하였으며, 세부 실험 설정과 결과는 'Data Augmentation - Answer 클래스 불균형 해소' 절에서 서술합니다.

4. 데이터 품질 점검 (문제 오류/라벨 이상)

EDA 과정에서 일부 샘플에서 문항 구성 자체가 깨져 있거나(예: 질문과 선택지의 주제가 불일치), 정답 라벨이 문항 내용과 맞지 않는 것으로 의심되는 사례를 발견하였습니다. 이러한 데이터 품질 이슈는 학습 과정에서 잘못된 패턴을 학습시키거나 성능을 왜곡할 수 있어, 실제로 성능에 영향을 미치는지 확인할 필요가 있다고 판단하였습니다. 이에 따라

후속 단계에서 AI 기반 전수 검증을 통해 문제 오류 및 답변(라벨) 이상 데이터를 식별하고, 해당 데이터를 제거했을 때 성능 변화가 있는지 비교 실험을 진행하였으며 세부 결과는 ‘Data Augmentation - 데이터 정제’ 절에서 서술합니다.

Data Preprocessing.

EDA 과정에서 파악된 문제 오류, 답변 이상 데이터를 제거하여 해당 데이터가 성능에 미치는 영향을 알아보기 위한 실험을 수행하였습니다.

Table 3. 데이터 정제 성능 비교 (Model: Qwen3-32B)

방법	Macro F1
원본 데이터	0.7970
답변 이상 데이터 제거 (-45개)	0.7914
문제 오류 데이터 제거 (-7개)	0.7896

Qwen3-32B 기준으로 데이터 정제 실험을 수행한 결과, 원본 데이터의 Macro F1은 0.7970이었습니다. 답변 이상 데이터 45개를 제거했을 때 0.7914로 감소했고, 문제 오류 데이터 7개를 제거했을 때는 0.7896으로 더 큰 폭으로 감소했습니다. 초기 예상과 달리 데이터를 제거한 조건에서 성능이 저하되었으므로, 노이즈로 판단된 데이터가 모델 학습에 일부 기여했거나 제거된 샘플 수가 적어 통계적 영향이 미미했을 가능성이 있다고 판단하였습니다. 따라서 이후 실험에서는 원본 데이터를 유지하였습니다.

Data Augmentation.

1. Answer 클래스 불균형 해소

EDA 과정에서 파악된 1번과 5번 정답 간의 클래스 불균형이 성능에 영향을 미치는지 파악하기 위해, 각각의 문제에 대해 모든 정답 번호 Case를 생성하는 Answer Shuffling을 사용하여 정답 라벨의 균형을 맞추었습니다. 그 결과 학습 데이터가 기존 2,031개에서 10,407개로 약 5배 증가되었고, 해당 데이터로 비교 실험을 수행하였습니다.

Table 4. 클래스 불균형 해소 성능 비교 (Model: Qwen3-32B)

방법	Macro F1
원본 데이터 (3 epochs)	0.7970
Answer Shuffling 증강 데이터 (3 epochs)	0.7787
Answer Shuffling 증강 데이터 (2 epochs)	0.7710

Answer Shuffling을 통한 데이터 증강 결과, 원본 데이터(3 epochs)의 Macro F1 0.7970에 비해 증강 데이터(3 epochs)는 0.7787로 저하되었습니다. 2 epochs로 감소시켰을 때도 0.7710으로 성능이 개선되지 않았습니다. 이는 원본 데이터의 정답 클래스 불균형이 모델 성능에 큰 영향을 미치지 않았음을

시사하며, 오히려 무리한 증강이 성능을 저하시킨 것으로 판단됩니다. 따라서 이후 실험에서는 원본 데이터를 유지하였습니다.

2. CoT 데이터 증강

Gemini-3.0 Flash 모델을 통해 train 데이터에 대한 해설(reasoning) 컬럼을 생성했습니다.

3. 공무원 시험 데이터 증강

9급 공무원 시험 국어와 한국사 분야의 문제 240개를 추가로 학습에 사용해보았습니다.

Table 5. 공무원 시험 데이터 성능 비교 (Model: DeepSeek-R1-Distill-Qwen-32B)

방법	성능
원본 데이터	0.7242
공무원 시험 증강 데이터	0.7602

공무원 시험 데이터 240개를 추가 학습한 결과, 원본 데이터의 성능 0.7242에서 0.7602로 향상되었습니다. 공무원 시험 데이터가 유사한 도메인 특성을 가져 성능 향상에 기여한 것으로 보이며, 외부 데이터 활용 가능성을 확인할 수 있었습니다.

Train.

1. Unsloth 적용하여 학습 가속

본격적인 학습 실험 시작에 앞서, 사전 양자화 모델 사용 및 학습 속도 단축을 위해 Unsloth 라이브러리를 적용한 베이스라인을 새로 구축하였습니다.

Table 6. Qwen3-32B 기준 모델 로드 시 필요 메모리

방법	메모리
기본 모델 로드 후 양자화	약 66 GB
사전 양자화된 모델 사용(4bit)	약 20 GB

사전 양자화된 모델을 사용한 결과 모델 로드 시 필요 메모리를 66GB에서 20GB로 단축하여 3배 이상 절약할 수 있었습니다.

Table 7. Qwen3-8B 기준 학습 소요시간

방법	소요시간
Unsloth 적용 전 (trl 사용)	약 5시간
Unsloth 적용 후	약 40분

Unsloth 적용 결과 학습 시간을 5시간에서 40분으로 단축하여 약 7배 이상 개선할 수 있었으며, 이를 통해 이후 실험 반복 속도를 크게 향상시킬 수 있었습니다.

2. 기본 데이터 학습

2.1 Pre-trained 모델 성능 비교

동일 파라미터로 LoRA, QLoRA 학습을 진행하여, Pre-trained 모델들의 성능을 비교해 보았습니다. 모델은 주로 성능이 좋다고 알려진 최신 모델들과 한국어 특화를 감안하여 국내에서 제작된 모델들을 선정하여 비교 실험을 수행하였습니다.

Table 8. Pre-trained LLM 성능 비교

Model	Macro F1
Qwen3-32B	0.7828
Qwen2.5-32B-Instruct	0.7735
A.X-4.0-Light	0.7646
Gemma-3-27B-it	0.7504
EXAONE-4.0-32B	0.7454
Mistral-Small-3.2-24B-Instruct-2506	0.7251
Phi-4 (14B)	0.7245
Qwen3-4B-Instruct-2507	0.6952
SOLAR-10.7B-Instruct	0.6298
LLAMA-VARCO-8B-Instruct	0.6072
Yi-34B-Chat	0.487

Qwen3-32B가 0.7828로 가장 높은 성능을 보였으며, Qwen2.5-32B-Instruct도 0.7735로 우수한 성능을 나타냈습니다. 특히 A.X-4.0-Light는 상대적으로 작은 모델 크기(7B)에도 불구하고 0.7646으로 예상 외로 높은 성능을 보였습니다. 이후 실험에서는 Qwen3-32B와 Qwen2.5-32B-Instruct를 주요 모델로 선정하여 진행하였습니다.

2.2 하이퍼파라미터 튜닝

학습 데이터로 달성할 수 있는 최고 점수와, 모델의 최적 하이퍼파라미터를 파악하기 위해 다양한 관점에서 실험을 진행하였습니다.

2.2.1 Epochs

과소적합 또는 과대적합의 가능성을 검증하기 위해 Qwen3-32B에 대해 5 epochs까지 학습을 진행하여 각 epoch별 성능을 비교하였습니다.

Table 9. Epoch 별 성능 비교 (Model: Qwen3-32B)

Checkpoint	Epochs	Macro F1
828	2	0.7756

3048	3	0.7828
4064	4	0.7813
5080	5	0.7793

3 epochs에서 Macro F1 0.7828로 가장 높은 성능을 보였으며, 4 epochs 이후로는 성능이 감소하는 경향을 보였습니다. 이는 3 epochs 이후 과대적합이 발생함을 시사하므로, 이후 실험의 기준 epochs를 3으로 설정하였습니다.

2.2.2 Batch size

학습 속도 단축을 위해 배치 사이즈를 증가시켜도 될지 모델 성능과의 관계를 알아보기 위해 실험을 진행하였습니다.

Table 10. Batch size 성능 비교 (Model: A.X-4.0-Light)

train_batch_size	gradient_accumulation_steps	Macro F1
1	1	0.7646
2	8	0.7368

배치 사이즈 증가로 예상보다 큰 폭의 성능 저하(0.7646에서 0.7368로 약 2.8% 감소)가 발생하였습니다. 따라서 배치 사이즈는 기존 설정을 유지하였습니다.

2.2.3 LoRA 계수

LoRA 계수 증가에 따른 성능 변화를 비교하였습니다.

Table 11. LoRA 계수 별 성능 비교

Model	LoRA r	LoRA alpha	Macro F1
Qwen3-32B	8	16	0.7828
Qwen3-32B	16	32	0.7970
Qwen3-32B	32	64	0.7766
Qwen2.5-32B-Instruct	8	16	0.7735
Qwen2.5-32B-Instruct	16	32	0.7881
Qwen2.5-32B-Instruct	32	64	0.7823

Qwen3-32B는 LoRA r=16, alpha=32에서 0.7970으로 최고 성능을 보였으며, r=32로 증가시켰을 때는 0.7766으로 성능이 저하되었습니다. Qwen2.5-32B-Instruct도 r=16, alpha=32에서 0.7881로 최고 성능을 나타냈습니다. 따라서 두 모델 모두 LoRA r=16, alpha=32를 최적 설정으로 선정하였습니다.

2.2.4 LoRA dropout, Learning Rate

LoRA dropout 및 Learning Rate 조정에 따른 성능을 비교하였습니다.

Table 12. dropout, LR 조정에 따른 성능 비교

Model	LoRA dropout	Learning Rate	Macro F1
Qwen3-32B	0	2e-5	0.7970
Qwen3-32B	0.1	2e-5	0.7925
Qwen3-32B	0.1	1.5e-5	0.7926

LoRA dropout과 Learning Rate 조정으로 성능이 저하되었으므로, 기존 설정(dropout=0, lr=2e-5)을 유지하였습니다.

3. CoT 데이터 학습

3.1 CoT 적용 실험 비교

CoT(Chain-of-Thought) 데이터 적용이 성능에 미치는 영향을 확인하기 위해, 동일한 학습 파라미터 설정을 유지한 상태에서 원본 데이터 학습과 CoT 데이터 학습 결과를 비교하였습니다.

또한 CoT 데이터 조건을 동일하게 유지한 상태에서 base 모델(Qwen3-32B)과 instruct 모델(Qwen2.5-32B-Instruct)을 비교하여, CoT 학습에서 instruct 튜닝된 모델 사용이 성능에 미치는 영향을 탐색적으로 확인하였습니다.

다만 모델 버전 차이가 함께 존재하므로, 성능 차이가 instruct 때문인지를 이번 비교만으로는 깔끔하게 분리해 말하기 어렵습니다.

Table 13. CoT 데이터 적용 실험 성능 비교

Model	Macro F1
Qwen3-32B (원본 데이터)	0.7828
Qwen3-32B (CoT 데이터)	0.7844
Qwen2.5-32B-Instruct (CoT 데이터)	0.7979

Qwen3-32B 기준으로 CoT 데이터를 적용했을 때 Macro F1-score가 0.7828에서 0.7844로 소폭 향상되었습니다.

동일한 CoT 데이터 조건에서 Qwen2.5-32B-Instruct 모델은 0.7979로 Qwen3-32B 대비 더 높은 성능을 보였습니다.

즉, 구버전(Qwen2.5)임에도 instruct 튜닝된 모델에서 더 높은 성능이 관찰되어, 이후 CoT 실험은 Qwen2.5-32B-Instruct를 기준 모델로 고정하여 진행하였습니다.

다만 Qwen3 대비 모델 버전 차이가 존재하므로, 본 결과를 instruct 효과로 단정하기보다는 실험 효율을 위한 모델 선택 근거로 활용하였습니다.

3.2 CoT 하이퍼파라미터 튜닝

CoT 학습의 특성을 고려하고, 과적합을 방지하기 위해 최적의 하이퍼파라미터를 탐색하는 실험을 진행하였습니다. 공정하

비교를 위해 모델·데이터·평가 조건은 동일하게 유지하고, 학습 설정만 변경하여 성능을 비교하였습니다

Table 10의 첫 번째 설정은 3.1에서 사용한 Qwen2.5-32B-Instruct(CoT 데이터) 기본 학습 설정(Macro F1=0.7979)으로, 이후 튜닝 결과 비교를 위한 기준선으로 사용하였습니다.

Table 14. CoT 학습 하이퍼파라미터 성능 비교
(Model: Qwen2.5-32B-Instruct)

LR	Epochs	Weight decay	LoRA r	LoRA alpha	Macro F1
2e-5	3	0.0	8	16	0.7979
1e-4	2	0.0	32	64	0.7843
1e-4	2	0.0	128	128	0.7982
1.3e-4	2	0.0	128	128	0.7850
1e-4	2	0.01	128	128	0.7998
1e-4	2	0.0	128	192	0.7939
1e-4	2	0.0	144	128	0.7888

Table 10 기준, weight decay=0.01을 적용한 설정이 Macro F1 0.7998로 가장 높은 성능을 기록하였습니다.

반면 learning rate=1.3e-4 설정은 0.7850으로 성능이 낮게 측정되었습니다. 또한 본 탐색 범위에서는 LoRA r/alpha 조합을 변경하더라도 성능이 일관되게 개선되는 패턴은 확인되지 않았습니다.

최종적으로 실험 결과를 바탕으로 CoT 학습의 최적 하이퍼파라미터를 아래와 같이 선정하였습니다.

- Learning Rate: 1e-4 (↑; CoT 학습 신호 강화 목적)
- Epochs: 2 (↓; CoT에서 과적합/패턴 고착 방지)
- Weight decay: 0.01
- LoRA r: 128 (↑; CoT reasoning 패턴 수용력 증가)
- LoRA alpha: 128 (↑; r에 맞춰 capacity 확장)

3.3 CoT 추론 입력 형식 정합

CoT 학습 과정에서 사용한 입력 형식과 추론 시 입력 형식이 불일치할 경우, 동일 모델이라도 평가 결과가 달라질 수 있어 추론 입력 포맷을 학습 설정과 동일하게 맞추는 실험을 진행하였습니다.

본 실험에서는 학습 파라미터를 고정한 상태에서, 추론 입력 형식을 Raw text에서 Chat template 적용 방식으로 정합하여 성능 변화를 비교하였습니다.

Table 15. CoT 추론 입력 형식 정합 (최적 하이퍼파라미터 세팅)

방법	Macro F1
----	----------

Raw text 추론	0.7998
Chat_template 적용 추론	0.8003

동일한 최적 하이퍼파라미터 세팅에서 Chat_template 적용 추론이 Macro F1 0.8003으로 소폭 높게 측정되었습니다.

Inference.

1. Self-Consistency 추론

추론 단계에서 애매한 문항에만 선택적으로 추가 추론 비용을 사용하는 구조를 도입했습니다.

- Confidence(margin) 계산 추가
- 제한적 Self-Consistency 적용
- Self-Consistency 비용 상한 설정

Table 16. Self-Consistency 추론 성능 비교

구분	Macro F1
Before	0.6660
After_ver1	0.6992
After_ver2	0.7093

불확실성 지표를 margin 외에 entropy, top1_prob까지 3개로 확장하며, 각각 지표의 임계값들과 SC 조건을 조절해가며 최적의 성능을 평가한 결과 After_ver2의 성능을 달성할 수 있었습니다.

2. 위키피디아 데이터 RAG

2.1 RAG 데이터셋 구축

2.1.1 구축 배경

기존 데이터셋 분석 결과, 주어진 지문 내 정보 외에도 특정 도메인에 대한 배경지식이 있어야만 해결 가능한 문항들이 확인되었습니다. 이에 따라 모델의 답변 정확도를 높이기 위한 외부 지식 베이스(External Knowledge Base) 구축이 필요하다고 판단하였습니다.

대회 규정상 수능 관련 데이터의 직접적인 활용이 제한되며, Agent Tool을 연동할 경우 의도치 않게 제한된 데이터가 유입될 가능성이 있습니다. 따라서 이를 방지하고 공신력 있는 범용 지식을 확보하기 위해, 한국어 데이터 중 가장 방대한 규모를 가진 위키피디아를 활용하기로 결정하였습니다.

2.1.2. 데이터 정제 전략

2025년 12월 16일 기준의 위키피디아 덤프 데이터를 사용하였으며, 데이터의 품질을 높이기 위해 아래와 같은 정제 과정을 수행하였습니다.

- 페이지 필터링: 전체 2,157,147건의 페이지 중 네임스페이스가 '문서'인 항목을 선별하였으며, 실질적인

정보량이 부족한 100자 미만의 문서와 리다이렉트 페이지를 제거하였습니다.

- 포맷 최적화: LLM의 가독성을 높이기 위해 XML 형태를 마크다운(Markdown)형식으로 변환하였습니다.
- 노이즈 제거: 파일 정보 및 불필요한 링크를 삭제하였습니다. 특히 링크와 수식의 구분 체계가 모호한 점을 고려하여, 수식 기호를 보존하기 위해 링크의 마지막 텍스트는 유지하는 전처리 로직을 적용하였습니다.

정제 결과 최종 565,484개의 페이지를 확보하였으며, 효율적인 관리를 위해 메타데이터(Title, Page ID, Length 등)를 포함하여 [HuggingFace](#)에 업로드하였습니다.

2.1.3 청킹 전략

활용 예정인 임베딩 모델(multilingual-e5-large-instruct)의 최대 토큰 제한(512)을 고려하여, 각 문서를 400자 단위로 분할하였습니다. 이때 마크다운 구조의 장점을 활용하여, 각 청크가 속한 섹션 정보를 header_path 컬럼에 명시함으로써 문맥 정보를 보존하였습니다.

생성된 총 4,322,674개의 청크는 한정된 컴퓨팅 자원(V100 1대, 사용인원 2명) 환경에서 스토리지 부족 문제를 야기할 수 있습니다. 이를 해결하기 위해 데이터를 [HuggingFace 스트리밍 모드로 업로드](#)하여 로컬 저장 공간의 제약을 극복하였습니다.

2.1.4 임베딩 + FAISS 인덱싱 전략

벡터 DB는 FAISS를 사용하였으며, 임베딩 모델로는 multilingual-e5-large-instruct를 채택하였습니다. 구축 과정에서 발생한 시간 및 자원 효율성 문제는 다음과 같이 해결하였습니다.

- 처리 속도 개선: 초기 실행 시 약 16~18시간이 소요될 것으로 예상되었으나, FP16 정밀도 활성화 및 배치 사이즈 조정(128 -> 512)을 통해 전체 작업 시간을 4~5시간 이내로 단축하였습니다.
- 메모리 및 저장 공간 관리: 인덱스 저장에 필요한 공간(25.35 GB) 대비 가용 환경이 협소한 점을 고려하여, mmap(Memory Mapping) 방식을 도입하였습니다. 이를 통해 데이터를 메모리에 전체 로드하지 않고 배치 단위로 읽어 FAISS 인덱스에 추가함으로써, 한정된 자원 내에서 인덱싱을 완수하였습니다.

작업 효율성을 높이기 위해 최종 생성된 [임베딩](#)과 [인덱스](#) 파일을 허깅페이스에 업로드하여 중복 작업을 방지하였습니다.

2.2 RAG 적용 실험

2.2.1 Confidence 기반 RAG

RAG를 무조건 적용하지 않고 모델의 confidence score 기반으로 “지식 부족 문제”만 선별하며 기존 추론 방식인 SC와 역할이 겹치지 않도록 분리하여 RAG 적용이 실제로 도움이

되는 문제만 대상으로 삼는 게이트를 구축하는 것을 목표로 하였습니다.

기본 구조

- Base Model: Qwen2.5-32B-Instruct (bnb 4bit)
- Adapter: qwen2.5-32b-qlora-15
- 추론 방식:
 - Logit-based next-token classification (1~5)
 - Optional Self-Consistency (1-token sampling, budgeted)

Confidence 정의

$conf_score = margin + toplp - entropy_norm$

기존 SC에서 사용하는 3개의 지표를 활용해 실험적으로 confidence score 및 RAG 사용 조건을 정의하였습니다.

Table 11. Confidence 기반 RAG 실험 비교

RAG	Macro F1
Base model(Qwen2.5-32B-Instruct-CoT)	0.8003
RAG_confidence_score	0.8018

2.2.2 Retriever 구조 변경

기본적으로 RAG가 발동될 때 문서 “원문 chunk”를 그대로 붙이지 않고 질문과 관련성이 높은 핵심 문장만 추출해 프롬프트에 넣어 문서 오염을 감소하도록 했습니다. 이를 통해 컨텍스트 길이 폭발 방지 및 정답 근거 집중도 향상을 목표로 했습니다.

다만 추출되는 문서 및 문장이 지문과 관련이 없는 noise로 작용하는 본질적인 문제가 있었습니다. 이를 위해 Retriever를 구축하는 구조를 변경했습니다.

기존 Retriever 구조

- [전체 문서 (~1GB+)]
- ↓ (offline)
- BM25 (question 중심 pruning)
- ↓
- subset (약 20%) - 이 부분에서 데이터 손실 발생
- ↓
- Dense embedding + FAISS
- ↓
- BM25 + Dense hybrid
- = (optional) rerank (bge-reranker-v2-m3)

변경된 최종 구조

- [전체 문서]
- ↓
- Chunking (기존 방식 유지)
- ↓

Dense embedding (multilingual-e5-base)

- ↓
- FAISS index
- ↓
- Inference-time:
 - Dense recall + BM25 sparse recall
 - = rerank (optional)

Table 11. Retriever 구조 변경 실험 비교

구분	Macro F1
Before	0.8018
After	0.7993

여러 파라미터 수정에도 불구하고 오히려 데이터가 늘어남이 noise 증가로 연결되며 성능이 하락하는 결과를 얻었다고 추측했습니다.

2.3.3 LLM Gate 기반 RAG

RAG의 적용 여부를 confidence 대신 open source 모델을 활용한 Gate 형태로도 실험해보았습니다.

Table 11. LLM Gate 기반 RAG 실험 비교

Gate 적용 구분	Macro F1
Before	0.8018
After	0.8046

(LLM Gate 기반 RAG 결과에 대한 간단한 코멘트)

2.3.4 RAG 데이터셋 변경

이외에도 원본 데이터의 한계로 위 문제가 발생하고 있을 수도 있다는 가정하에 Document를 구축하는 데이터를 기존 kowiki-cleaned에서 namuwiki-sentences로 변경을 했습니다.

Table 11. RAG 데이터셋 변경 실험 비교

Data	Macro F1
kowiki-cleaned	0.8046
namuwiki-sentences	0.8066

결과적으로 단일 모델 기준 0.8003에서 0.8066으로 향상되었지만, 실제 적용된 문서를 분석한 결과 80% 이상이 관련이 없는 내용이 포함되었으며, RAG를 사용해 만든 추출물을 ensemble에 포함했을 때, 성능이 급격히 하락하는 현상을 발견해 RAG 실험을 중단했습니다.

3. Llama Cpp 활용 Zero-shot 추론

30B 급의 MoE 모델은 QLoRA 학습이 불가능하여, LlamaCpp를 사용하여 학습하지 않은 상태의 모델로 제로샷 추론을 시도하였습니다.

프롬프트에는 수능 최상위권 수험생 역할 부여, 지문 우선주의 및 오답 소거 전략, JSON 형식 강제 출력 등의 지시문을 포함하였습니다. 응답 형식이 맞지 않는 경우 최대 5회까지 형식 안내 프롬프트를 추가하여 재추론하였습니다. 그럼에도 응답을 얻지 못한 경우, 기본값(1)으로 처리하거나 다른 모델의 우수한 추론 결과로 대체하는 두 가지 폴백 전략을 적용하여 성능을 비교하였습니다.

Table 11. RAG 데이터셋 변경 실험 비교

Model	Macro F1
Qwen3-32B LoRA (기준선)	0.7970
Qwen3-30B-A3B-Thinking-2507 (기본값 사용)	0.5167
Qwen3-30B-A3B-Thinking-2507 (추론 결과 대체)	0.5893

제로샷 추론 결과는 기준선 대비 크게 낮은 성능을 보였습니다. 기본값(1)으로 폴백한 경우 0.5167, 다른 모델의 추론 결과로 대체한 경우 0.5893으로, 폴백 전략을 통해 약간의 성능을 개선할 수 있었습니다.

이를 통해 학습 없이 프롬프트 엔지니어링만으로는 충분한 성능을 달성하기 어려우며, Few-shot 프롬프팅이나 파인튜닝이 필요함을 느꼈습니다. 다만 제한된 시간으로 인해 추가 실험은 수행하지 못하였습니다.

Ensemble.

1. 확률 평균 앙상블

개별 실험 결과의 편차를 완화하고 예측의 안정성을 높이기 위해, 각 모델이 출력한 클래스별 확률(softmax probability)을 평균내는 확률 평균 앙상블(soft voting)을 적용하였습니다.

구체적으로 동일 샘플에 대해 모델별 확률 분포를 집계한 뒤, 클래스별 평균 확률이 가장 큰 라벨을 최종 예측으로 선택하도록 구성하였으며, 단순 다수결(hard voting) 대비 모델의 확신도 정보까지 반영할 수 있도록 설계하였습니다.

2. 앙상블 조합 및 결과

약 30개 이상의 앙상블 조합을 실험하였으며, 최고 성능 조합 1개와 이와 다른 구성의 조합 중 성능이 우수한 1개를 선정하여 총 2개를 최종 제출에 사용하였습니다.

2.1 soft_ensemble_3_

Table 11. soft_ensemble_3_ 조합

Model	Macro F1
NLP-07-ODQA/Qwen2.5-32B-Instruct-bnb-4bit_8	0.7982
NLP-07-ODQA/Qwen2.5-32B-Instruct-bnb-4bit_15	0.8003

NLP-07-ODQA/Qwen3-32B-bnb-4bit-CoT_2	0.7844
--------------------------------------	--------

3개 모델의 확률을 평균내는 soft voting을 적용한 결과, 앙상블 Macro F1은 0.8297을 달성하였습니다.

2.2 ensemble_6

Table 11. ensemble_6 조합

Model	Macro F1
NLP-07-ODQA/qwen2.5-32b-it-qlora-v4.1	0.7881
NLP-07-ODQA/qwen2.5-32b-it-qlora-v4.2	0.7823
NLP-07-ODQA/Qwen2.5-32B-Instruct-bnb-4bit_8	0.7982
NLP-07-ODQA/Qwen2.5-32B-Instruct-bnb-4bit_15	0.8003
NLP-07-ODQA/qwen3-32b-qlora-v6.1	0.7925
NLP-07-ODQA/qwen3-32b-qlora-v7.1	0.7896

6개 모델의 확률을 평균내는 soft voting을 적용한 결과, 앙상블 Macro F1은 0.8257을 달성하였습니다.

2.3 앙상블 결과

Table 11. 최종 제출 2개 결과

Ensemble	Macro F1	Macro F1 (최종)
soft_ensemble_3_	0.8297	0.7364
ensemble_6	0.8257	0.7684

soft_ensemble_3_은 앙상블 단계에서 0.8297으로 가장 높은 점수를 기록했으나, 최종 제출 결과에서는 ensemble_6이 0.7684로 더 좋은 성능을 보였습니다.

IV. Conclusion

최종 솔루션.

최종 제출에는 ensemble_6을 사용하였습니다. Qwen2.5와 Qwen3 모델의 6개 버전을 soft voting으로 결합하여 각 모델의 예측 확률값들의 평균을 계산하고 최종 답변을 결정합니다.

최종 결과.

1. Public 리더보드

순위	팀이름	팀멤버	MACRO_F1	제출횟수	최종 제출
내등수 7	NLP-07		0.8297	147	1w

Figure 3. Public 리더보드

2. 최종 리더보드

순위	팀이름	팀멤버	MACRO_F1	제출횟수	최종 제출
내등수 5	NLP-07		0.7684	147	1d

Figure 4. 최종 리더보드

분석.

개선 필요 사항 (Areas for Improvement).

본 연구에서 구축한 시스템은 대회 환경의 특수성을 고려하여 설계되었습니다. 실시간 응답성이 중요한 실제 비즈니스 모델에 적용하기에는 속도 측면의 개선이 추가로 필요할 것으로 판단됩니다.

개인 회고 - **T8001** 가을

개인 회고

수능형 문제 풀이 모델 생성

가을(T8001)

1. 학습 목표

본 프로젝트에서 저의 목표는 단순 지문 기반 답변을 넘어, 외부 지식 베이스를 연동하여 모델의 배경지식을 보완하는 **RAG** 파이프라인을 구축하는 것이었습니다. 특히 자원 제약 환경 하에서 대규모 데이터 처리 역량을 기르고, 추론 로직의 고도화(Agentic Workflow)를 통해 객관식 문항의 정답률을 극대화하는 데 집중하였습니다.

2. 기술적 도전과 해결 과정

2.1 자원 제약 극복을 위한 공학적 접근

가용 디스크 공간(13GB) 및 시스템 RAM의 한계를 극복하기 위해 메모리 매핑(**mmap**) 기술을 도입하였습니다. 약 17.7 GB 규모의 FAISS 인덱스를 메모리에 전체 로드하지 않고 배치 단위로 참조함으로써, 시스템 RAM 사용량을 약 80% 이상 절감하였습니다. 이를 통해 V100 32GB 환경에서 **Qwen3-14B** 모델과 검색 엔진을 동시에 안정적으로 구동하는 최적의 자원 배분을 달성하였습니다.

2.2 512 토큰 제한 대응 및 계층적 청킹

multilingual-e5-large-instruct 모델의 입력 제한을 고려하여, 마크다운 헤더 구조를 보존하는 계층적 청킹 전략을 수립하였습니다. 섹션 정보를 메타데이터로 관리하고 400자 단위로 재분할하여 정보 유실을 방지하였으며, FP16 정밀도 적용 및 배치 사이즈 조정을 통해 임베딩 속도를 4배 이상 가속화하였습니다.

2.3 Agentic RAG 로직의 단계적 고도화

단순 검색의 한계를 극복하기 위해 추론 로직을 점진적으로 개선하였습니다.

- 통합 쿼리에서 선택지별 쿼리로 전환: 초기에는 지문 전체를 아우르는 통합 쿼리를 사용했으나, 각 선택지의 정오답을 개별적으로 검증하기 위해 선택지별 맞춤형 쿼리 생성 로직을 도입하였습니다.
- 신뢰도 기반 분류 시스템: 모델의 답변 신뢰도(Confidence)를 수치화하여 '명확히 맞음(0.8 이상)', '명확히 틀림(0.2 이하)', '모호함'으로 분류하는 의사결정 프로세스를 구축하였습니다. 그 결과 Macro F1-score를 **0.4574**에서 **0.4637**로 향상시켰습니다.

3. 한계점 및 아쉬운 점

max_seq_length를 8192로 확장하거나 지문 참조 범위를 전체로 늘리는 등 다양한 파라미터 튜닝을 시도했으나, 답변 분포의 유의미한 변화를 이끌어내지 못했습니다. 이는 단순한 하이퍼파라미터의 문제가 아닌, 검색된 문서의 품질이나 프롬프트의 구조적 한계일 가능성이 높다고 판단됩니다.

실험 후반부에서 추론 과정의 병목과 모델의 사고 과정을 더 정밀하게 추적하기 위해 **LangSmith** 도입을 추진했으나, 대회 일정상 충분히 활용하지 못한 점이 아쉽습니다. 향후에는 파이프라인 설계 초기 단계부터 관측성(Observability) 도구를 결합하여 데이터 흐름을 실시간으로 모니터링할 필요성을 절감하였습니다.

4. 총평 및 향후 과제

이번 프로젝트를 통해 한정된 인프라 자원을 효율적으로 관리하는 법을 배웠으며, RAG 시스템에서 데이터 전처리(Cleaned Wiki)와 추론 전략(Agentic RAG)이 성능에 미치는 영향력을 체감하였습니다. 차후에는 성능 향상을 위해 미뤄두었던 리랭커(Re-ranker)의 경량화 적용과 선택지별 교차 검증 로직을 더욱 정교하게 다듬어보고 싶습니다.

개인 회고 - **T8078** 박신지

개인 회고

수능형 문제 풀이 모델 생성

박신지(T8078)

1. 학습 목표

이번 프로젝트에서 나의 주요 학습 목표는 모델 성능을 올리기, 프롬프트 튜닝을 통한 inference 성능 높이기였다.

2. Distill Qwen2.5 32B 모델 실험

처음에는 14B 모델을 통해 베이스라인을 짰다. 그러나 성능이 좋지 않아 32B 모델을 정하기로 결정했다. 32B 모델은 뛰어난 추론 능력을 가지고 있으나 v100 자원으로 바로 가져오기엔 한계가 있었다. 따라서 unsloth를 이용한 실험을 진행했다. 초반에는 답만 주고 맞추는 모델 학습을 진행했고, 그 후 파라미터 조정을 통해 튜닝을 했다. 마지막으로 CoT 데이터를 증강해서 CoT 학습을 진행했으나 성능은 기본이 제일 좋았다.

3. 데이터 증강

데이터 증강은 총 두 번 했다. 처음엔 Gemini-Flash 3.0을 통해 CoT 데이터를 만들었다. CoT 데이터는 train 데이터를 보고 이 지문과 답에 대한 해설을 만드는 식으로 진행했다. 그러나 이를 통해 test 데이터에 추론을 해보니 추론 시간이 너무 길어지는 문제가 발생했다. max token 수를 512로 줄인 후에는 추론 시간이 줄었으나 CoT 데이터는 1000자 정도인데 max token 수는 그보다 턱없이 적은 탓에 성능이 나오지 않았던 것으로 추측된다.

다음으로는 공무원 시험 데이터 200개 정도를 증강했다. 이는 소폭의 성능 향상을 보였다.

4. 개선 시도 및 결과

max token 수를 512로 줄이면서 해설이 중간에 잘리는 문제도 있었다. 이 경우에는 프롬프트 튜닝을 통해서 최대한 300자 이내로 해설을 끝내라고 명령했다. 또한 이렇게 했음에도 잘렸을 경우엔 그런 지문들만 찾아서 1024자로 늘리는 실험까지 진행했다.

CoT 실험의 경우에는 정답이 1로 많이 편향된 모습을 보였다. 이는 train 데이터에서 1이 많았기 때문에 그런 것 같다는 추측이 있었다. inference에서만 1 편향을 줄이려는 노력을 했는데, train 데이터를 수정해서 편향을 없애는 것도 방법이 됐을 수 있을 것 같다.

5 다음 프로젝트에서의 개선 방향

CoT 데이터를 만들 때 처음부터 max token 수를 적게 했으면 좋았을 것 같다는 아쉬움이 남는다. 이런 부분에서 더 체계적으로 계획하고 실험했다면 자원의 낭비가 없었을 수 있을 것 같다.

개인 회고 - **T8164** 이형석

수능형 문제 풀이 모델 생성

이형석(T8164)

1. 학습 목표

본 프로젝트의 핵심 목표는 수능형 객관식 문제 환경에서 안정적인 일반화 성능을 확보하는 것이었다. 단순히 단일 지표를 높이는 것이 아니라, macro F1 기준으로 모든 선택지에 대해 균형 잡힌 예측을 수행하는 모델을 구축하는 데 초점을 맞추었다. 특히 대형 언어 모델을 활용한 실험에서 특정 학습 전략이나 추론 기법이 오히려 분포 붕괴를 유발하는 사례를 경험하면서, “성능 향상처럼 보이는 변화”와 “실제 제출 성능 개선”을 명확히 구분하는 것을 중요한 목표로 삼았다. 이를 위해 RAG, ORPO/DPO, SC 등 다양한 기법을 무작정 적용하기보다, 실제 데이터와 문제 특성에 적합한지를 검증하며 점진적으로 접근하고자 했다.

2. 기술적 도전과 해결 과정

가. **RAG:** 적용 범위와 데이터 적합성 검증

문제: 작년 대회 분석을 통해 RAG가 성능에 도움이 되지 않거나 오히려 하락을 유발할 수 있다는 점을 인지했음에도, 실제 실험에서는 RAG 구축에 많은 시간을 할애했다. 팀원이 정제한 위키피디아 데이터를 기반으로 TF-IDF와 FAISS를 결합한 하이브리드 검색 구조를 사용했으나, 모든 문제에 RAG를 적용할 경우 노이즈가 과도하게 유입되는 문제가 발생했다.

해결: 모든 test 데이터에 RAG를 적용하지 않고, 기존 SC에서 활용하던 세 가지 지표를 기반으로 confidence_score를 산출해 RAG 적용 여부를 결정하는 구조를 설계했다. 또한 로그를 통해 실제 869개 데이터 중 RAG가 몇 퍼센트에서 동작하는지, SC와 얼마나 겹치는지를 점검했다.

나. **Model & Inference:** 다양한 게이팅과 학습 기법 실험

문제: RAG를 포함한 단일 모델 성능은 일정 수준까지 상승했으나, ensemble 단계에서 RAG를 포함시키면 public 성능이 일관되게 하락했다.

해결: 나무위키 데이터 추가, 임베딩 방식 변경, confidence_score 대신 Qwen2.5-32B를 활용한 게이트 결정 등 다양한 변형을 시도했다. 동시에 ORPO, DPO와 같은 preference 기반 학습을 적용해 정답·오답 간 margin을 확장하는 실험도 병행했다. llama.cpp와 unsloth를 활용해 제한된 자원 환경에서 대형 모델을 효율적으로 학습·추론하는 파이프라인을 구축했다.

3. 결과 및 인사이트

실험 결과 단일 모델 기준으로는 0.8066이라는 성능을 얻었으나, 최종적으로는 ensemble에 RAG를 포함할 경우 성능 하락을 피할 수 없었다. 특히 실제 프롬프트에 포함된 문서를 직접 확인한 결과, 약 80%의 문서가 문제와 직접적인 관련이 없는 노이즈였고, 유의미한 문서는 20% 수준에 불과했다. 이를 통해 **RAG**의 핵심은 검색 알고리즘이 아니라 데이터의 적합성과 **recall**이라는 점을 명확히 인식하게 되었다. 또한 ORPO와 같은 강화 학습 계열 기법은 적절히 사용하면 효과가 있으나, 과도할 경우 분포 쏠림과 캘리브레이션 붕괴를 유발할 수 있다는 점도 중요한 인사이트로 남았다.

4. 한계점 및 아쉬운 점

가장 아쉬운 점은 RAG 구축 단계에서 데이터 자체에 대한 검증보다 방법론과 구조 설계에 지나치게 집중했다는 것이다. faiss, bm25, tf-idf와 같은 기법보다 먼저 문서의 실제 recall과 문제 적합성을 점검했어야 했다. 또한 작년 대회 팀들의 피드백을 더 적극적으로 반영했다면, RAG에 대한 판단을 더 빠르게 내릴 수 있었을 것이라 생각한다. ORPO, DPO, PPO와 같은 강화 학습 기법 역시 부분 적용이나 조건부 적용 등 더 정교한 실험을 진행하지 못한 점이 아쉽다. 그럼에도 이번 프로젝트를 통해 대형 모델,

데이터 증강, 자원 효율적 학습·추론에 대한 이해를 크게 확장할 수 있었으며, 이는 이후 프로젝트에서 보다 전략적인 선택을 가능하게 하는 중요한 경험으로 남았다.

개인 회고 - **T8091** 박희권

개인 회고

수능형 문제 풀이 모델 생성

박희권(T8091)

1. 학습 목표

이번 프로젝트에서는 대규모 언어 모델의 미세조정과 앙상블을 통해 다지선다형 질문 답변 시스템의 성능을 극대화하는 것을 목표로 했습니다. 구체적으로는 EDA를 통한 데이터 분석, 최적 하이퍼파라미터 탐색, 그리고 다양한 모델 조합 앙상블을 진행하였습니다.

2. 주요 시도 및 성과

프로젝트 기반 구축 및 베이스 모델 탐색

프로젝트 초기에 Unsloth 라이브러리를 적용하여 베이스라인을 재구축했습니다. 사전 양자화 모델 사용으로 모델 로드 시 필요 메모리를 66GB에서 20GB로 단축했고, 학습 시간을 5시간에서 40분으로 약 7배 개선하여 이후 실험 반복 속도를 크게 향상시킬 수 있었습니다. 이후 Qwen3-32B, Qwen2.5-32B-Instruct, A.X-4.0-Light 등 최신 모델들을 체계적으로 비교하여 최적 베이스 모델을 선정했습니다.

EDA 및 데이터 분석

초기 단계에서 정답 클래스 분포를 분석한 결과 1번 정답 948개, 5번 정답 31개로 심한 불균형을 발견했습니다. 이를 해소하기 위해 Answer Shuffling을 통한 데이터 증강을 시도했으나, 오히려 성능이 저하되어 원본 데이터 유지가 최적임을 확인했습니다. 또한 데이터 정제 실험을 통해 AI 기반으로 식별한 문제 오류 및 라벨 이상 데이터를 제거한 경우에도 성능이 저하되어, 해당 데이터들이 모델 학습에 일부 기여함을 알 수 있었습니다.

하이퍼파라미터 최적화

Epochs, Batch size, LoRA 계수 등을 체계적으로 조정하여 Qwen3-32B는 $r=16$, $\alpha=32$, 3 epochs에서 0.7970 성능을 달성했습니다.

모델 앙상블

프로젝트 후반에는 Qwen2.5와 Qwen3의 6개 모델 조합에 soft voting을 적용하여 Macro F1 0.8257을 달성했습니다. 이는 개별 모델의 최고 성능(0.8003)을 크게 상회하는 결과로, 다양한 모델의 강점을 효과적으로 결합할 수 있음을 보여줍니다.

3. 아쉬운 점

첫째, LlamaCpp를 활용한 Zero-shot 추론 실험을 충분히 진행하지 못한 점입니다. 해당 솔루션 구축이 늦어져서 프롬프트 엔지니어링과 폴백 전략을 다양하게 탐색하지 못했고, 다양한 MoE 모델들도 충분히 실험해보지 못했습니다. 충분한 시간이 있었다면 더 많은 모델을 시도하고 깊이 있게 실험해볼 수 있었을 것 같습니다.

둘째, 제한된 서버 환경으로 인한 어려움입니다. 라이브러리 설치 중 서버 다운이나 저장 공간 부족으로 인해 사용하고 싶었던 모델들을 시도하지 못한 경우가 있었습니다. 더 나은 인프라 환경이 있었다면 더 다양한 모델과 방법론을 실험할 수 있었을 것 같습니다.